📓 엔드투엔드 팩터 파이프라인 요약

(Code → Investment Process 매핑)

---

1 팩터 데이터베이스 구축

개요

- 종목 단위 **Point-in-Time(PIT)** 팩터 데이터베이스를 입력으로 사용
- 학술·실무 근거에 기반한 다수(200+) 팩터를 사전에 정의 및 축적
- 각 팩터는 **스타일 단위(Valuation, Momentum, Quality, Growth 등)**로 분류

코드 상 구현

- 입력 데이터
  - `data/{benchmark}_{start_date}_{end_date}.parquet` (SQL Server DB 에서 받은 상태 그대로 저장된 파일)
    - 종목 × 날짜 × 팩터 값 (`gvkeyiid, ticker, isin, ddt, sec, factorAbbreviation, val`)
  - `data/factor_info.csv`
    - 팩터 메타 정보 (팩터명, 스타일, 정렬방향)
- 본 코드는 **팩터 DB를 새로 구축하지 않고**,
  구축된 PIT 데이터베이스를 **분석 파이프라인의 입력값**으로 사용

---

2 팩터 후보군 선정

(2-1) 백테스트 기간 설정

- 분석 시작 시점: **2018년**
- 코드 구현
  - `_ls_portfolio()` 내 `ddt >= 2017-12-31`
  - → 2018년부터 실질적인 성과 반영

(2-2) 팩터별 5분위(Quintile) 포트폴리오 구성

- **핵심 함수:** `_assign_factor()`
- 절차
  - 종목별 팩터 값에 **1개월 래그 적용** (전월 값으로 당월 투자)
  - 동일 날짜·동일 섹터 내에서 팩터 값 순위 산정
  - 순위를 **백분위(0~100)**로 변환
  - 백분위를 **5분위(Q1~Q5)**로 구간화
- 결과
  - 각 팩터별 Q1~Q5 분위 포트폴리오 월간 수익률 산출

(2-3) 비투자 섹터 결정

- **핵심 함수:** `_filter_grouped()`
- 절차
  - 섹터별 팩터 스프레드 계산
    - `Spread = Q1 - Q5`
  - **스프레드가 음(-)인 섹터는 해당 팩터에서 제외**
- 목적
  - 구조적으로 팩터 성과를 훼손하는 섹터 제거
  - 팩터의 경제적 직관 유지

(2-4) 투자 대상 분위(롱/숏) 결정

- `_filter_grouped()`에서 재계산
- 절차
  - 섹터 제거 후 분위별 평균 수익률 재산출
  - Q1–Q5 평균 스프레드를 기준으로 임계값 설정
  - 각 분위를
    - 롱(+1)
    - 중립(0)
    - 숏(-1) 로 재분류
- 특징
  - 단순히 Q1=롱, Q5=숏이 아닌 **성과 기반으로 투자 대상 분위 선택**

(2-5) 팩터 스프레드 수익률 측정

- **핵심 함수 흐름**
  - `_ls_portfolio()`
    - 롱/숏 종목군 구성
    - 분위 내 **동일가중 포트폴리오**
  - `_vectorized_bt()`
    - 리밸런싱 반영
    - 턴오버 계산
    - 거래비용(30bp) 차감
  - `_aggregate_returns()`
    - 팩터별 **월간 롱–숏 스프레드 수익률** 생성

(2-6) 팩터 후보군 최종 선정

- **핵심 함수:** `_generate_meta()`
- 절차
  - 팩터별 월간 스프레드 수익률 행렬 구성
  - 연환산 수익률(CAGR) 계산
  - 스타일 내 / 전체 랭킹 산출
  - **CAGR 기준 상위 50개 팩터 선정**
  - 동시에 **하락 국면 상관관계(Downside Correlation)** 계산

---

3 최종 팩터 선정 및 비중 결정

(3-1) 스타일별 최상위 팩터 선정

- 스타일별 CAGR 기준 **1위 팩터를 메인 팩터로 선정**
- 목적
  - 각 스타일의 대표성 확보
  - 스타일 간 중복 최소화

(3-2) 보완 팩터 선정 및 2-팩터 믹스

- **핵심 함수:** `_get_wgt()`
- 절차
  - 메인 팩터 대비
    - 성과(CAGR)
    - 하락 상관관계 를 함께 고려
  - 보완 효과가 큰 서브 팩터 후보군 도출
  - 메인–서브 팩터 조합에 대해
    - 비중 0~100% 그리드 탐색
    - CAGR 및 MDD 평가
- 결과
  - **팩터 간 중복을 줄이면서 성과·안정성 개선**

(3-3) 스타일 제약 하 최적 비중 결정

- **핵심 함수:** `random_style_capped_sim()`
- 절차
  - 몬테카를로 방식으로 다수의 랜덤 포트폴리오 생성
  - 스타일별 비중 합계가 **최대 25%**를 넘지 않도록 제약
  - 각 포트폴리오의
    - CAGR
    - MDD 를 동시에 평가
- 결과
  - 스타일 분산을 유지한 **최적 팩터 비중 구조 도출**

---

4 MP 구성 및 매매 집행

(4-1) 종목별 최종 비중 산출

- 각 팩터 비중을 종목 수준으로 전개
- 방식
  - 롱/숏 종목군 내 동일가중
  - 팩터 비중만큼 스케일링
- 결과
  - 종목별 오버웨이트 / 언더웨이트 비중 산출

(4-2) Model Portfolio(MP) 구성

- 여러 팩터에서 계산된 종목 비중을 합산
- 팩터 통합 포트폴리오 = **MP**
- MP는 단일 스타일이 아닌 **팩터 집합의 가중 평균**

(4-3) 결과물 산출

- 종목 × 팩터 × 스타일 구조의 최종 가중치 패널 생성
- CSV 형태로 출력
  - 팩터별
  - 스타일별
  - MP 기준
- 포트폴리오 실행 시스템(BM/Optimizer)에 바로 연동 가능

(4-4) 실제 매매 집행

- 본 코드는 **Model Portfolio 산출까지 담당**
- 이후 단계
  - Benchmark 대비 Tracking Error 점검
  - Bloomberg Optimizer 등을 활용한 실제 매매 집행
- 실행은 별도의 운용·트레이딩 인프라에서 수행

✅ 전체 프로세스 요약

단계	목적
팩터 DB 구축	PIT 기반 종목·팩터 데이터 확보
후보군 선정	분위·섹터·스프레드 기반 성과 검증
최종 팩터 선정	스타일 대표성 + 상관관계 고려
비중 결정	스타일 제약 하 최적화
MP 구성	종목별 최종 투자 비중 산출

📎 Appendix: 함수별 Input / Output 정리 (I/O Reference)

> 아래는 본 코드에서 정의된 주요 함수들을 기준으로, **입력(필수 컬럼/형식)**과 **출력(산출물)**을 정리한 섹션입니다.
> (실제 구현/타입힌트/사용 흐름을 기준으로 작성)

`_rank_to_percentile(series: pd.Series) -> pd.Series`

- **Input**

    - `series`: `pd.Series`
        - 보통 `rank()` 결과(1~n)
- **Output**
    - `pd.Series`: 0~100 스케일 백분위 점수
    - 단, `n <= 10`이면 전부 `NaN`

`_n_quantile_label(score: float|int, n: int = 5) -> str|float`

- **Input**
    - `score`: 0~100 범위의 백분위 점수
    - `n`: 분위 개수 (기본 5)
- **Output**
    - `"Q1" ~ "Qn"` 라벨 문자열
    - 범위 밖/NaN이면 `np.nan`

`_add_initial_zero(series: pd.DataFrame) -> pd.DataFrame`

- **Input**
    - `series`: `pd.DataFrame`
        - **DatetimeIndex**를 가진 시계열 데이터프레임(예: spread)
- **Output**
    - `pd.DataFrame`
        - 첫 관측월의 **전월 인덱스**를 하나 추가하고 그 값을 `0`으로 삽입한 뒤 정렬된 결과

`_assign_factor(abbv: str, order: int, fld: pd.DataFrame, m_ret: pd.DataFrame)`

`-> Tuple[sector_ret, quantile_ret, spread, merged] | (None, None, None, None)`

- **Input**
    - `abbv`: 팩터 약어(컬럼명으로 사용)
    - `order`: 정렬 방향 (코드상 `ascending=bool(order)`로 사용)
    - `fld`: 특정 팩터의 원시 데이터프레임(이미 factorAbbreviation 기준 필터된 상태)
        - **필수 컬럼(예상)**: `["gvkeyiid","ticker","isin","ddt","sec","country","factorAbbreviation","val"]`
    - `m_ret`: 시장수익률(M_RETURN) 데이터프레임(미리 추출된 상태)
        - **필수 컬럼(예상)**: `["gvkeyiid","ticker","isin","ddt","sec","country","M_RETURN"]`
- **Output**
    - `sector_ret: pd.DataFrame`
        - 섹터×분위(Q1~Q5) 수익률 요약(가공된 형태)
    - `quantile_ret: pd.DataFrame`
        - 날짜×분위(Q1~Q5) 월간 평균 수익률
    - `spread: pd.DataFrame`
        - 날짜 인덱스, 열=`abbv`, 값= `Q1 - Q5` 스프레드 시계열 (+ 초기 0 삽입)

- `merged: pd.DataFrame`
  - 종목 단위 병합/가공 데이터(팩터값, M_RETURN, rank/percentile/quantile 포함)
- 히스토리가 매우 짧으면 `(None, None, None, None)` 반환

`_filter_grouped(list_abbrs, list_names, list_styles, list_data)`

`-> (kept_abbr, kept_name, kept_style, kept_idx, dropped_sec, new_raw)`

- **Input**
  - `list_abbrs`: 팩터 약어 리스트
  - `list_names`: 팩터 이름 리스트
  - `list_styles`: 스타일 이름 리스트
  - `list_data`: 각 팩터별 결과 리스트
    - 원소 형태: `(sector_ret, quantile_ret, spread, merged)` (=`_assign_factor` 반환)
- **Output**
  - `kept_abbr: List[str]` / `kept_name: List[str]` / `kept_style: List[str]`
    - 섹터 필터 후 살아남은 팩터 메타
  - `kept_idx: List[int]`
    - 원래 리스트에서 유지된 팩터 인덱스
  - `dropped_sec: List[List[str]]`
    - 팩터별로 제거된 섹터 리스트
  - `new_raw: List[pd.DataFrame]`
    - 섹터 필터 반영 후 **종목 단위 raw 데이터**
    - 추가 컬럼 포함: `label` (롱/중립/숏: +1/0/-1)

`_ncorr(df: pd.DataFrame, min_obs: int = 20) -> pd.DataFrame`

- **Input**
  - `df`: 월간 수익률 행렬(행=날짜, 열=팩터/스타일)
  - `min_obs`: 최소 표본 수(기본 20)
- **Output**
  - `pd.DataFrame`: 열×열 상관행렬
    - 각 열 `col`에 대해 `df[col] < 0`인 구간만 골라 상관 계산 (표본 부족 시 NaN)

`_ls_portfolio(data_raw: pd.DataFrame) -> Tuple[pd.DataFrame, pd.DataFrame]`

- **Input**
  - `data_raw`: 종목 단위 데이터(라벨 포함)
    - **필수 컬럼(예상)**: `["ddt","gvkeyiid","label","M_RETURN", ...]`
- **Output**
  - `(raw_df_l, raw_df_s)`:
    - `raw_df_l`: 롱 종목군 데이터프레임 (`signal=="L"`)
    - `raw_df_s`: 숏 종목군 데이터프레임 (`signal=="S"`)
  - 내부에서 생성되는 주요 컬럼
    - `signal`: L/N/S
    - `num`: 날짜×signal별 종목수

        - `wgt_rtn`: 수익률 계산용 비중(동일가중 기반)
        - `wgt_tvr`: 턴오버 계산용 비중(절대값)

`_vectorized_bt(port_raw: pd.DataFrame, abbr_nms: str, cost_bps: float = 30.0)`

`-> Tuple[gross_df, net_df, tf_df]`

- **Input**
    - `port_raw`: 롱 또는 숏 포지션 raw (=`_ls_portfolio` 결과 중 하나)
        - **필수 컬럼(예상)**: `["ddt","gvkeyiid","wgt_rtn","wgt_tvr","M_RETURN"]`
    - `abbr_nms`: 결과 컬럼명(팩터 약어)
    - `cost_bps`: 거래비용(bps)
- **Output**
    - `gross_df: pd.DataFrame`
        - 날짜별 gross return (열=`abbr_nms`)
    - `net_df: pd.DataFrame`
        - 날짜별 net return = gross - trading_friction
    - `tf_df: pd.DataFrame`
        - 날짜별 trading_friction(거래비용) 시계열

`_aggregate_returns(data_raw: List[pd.DataFrame], abbr_nms: List[str])`

`-> Tuple[df_grs, df_net, df_trc]`

- **Input**
    - `data_raw`: 팩터별 종목 raw 데이터 리스트(라벨 포함)
    - `abbr_nms`: 해당 raw에 대응하는 팩터 약어 리스트
- **Output**
    - `df_grs`: 팩터별 gross return 매트릭스(열=팩터)
    - `df_net`: 팩터별 net return 매트릭스(열=팩터)
    - `df_trc`: 팩터별 거래비용 매트릭스(열=팩터)

`_generate_meta(abbrs, names, styles, data)`

`-> Tuple[ret_df, negative_corr, meta]`

- **Input**
    - `abbrs/names/styles`: 팩터 메타 리스트
    - `data`: 팩터별 종목 raw 데이터 리스트(=`_filter_grouped` 산출 `cleaned_raw`)
- **Output**
    - `ret_df: pd.DataFrame`
        - 월간 net return 매트릭스(행=월, 열=팩터)
        - 유효성 필터(0 수익률 과다) 반영
        - 상위 50개 팩터로 축소된 결과가 반환됨
    - `negative_corr: pd.DataFrame`

        - downside(음수 구간) 상관행렬 (`_ncorr(ret_df)`)
    - `meta: pd.DataFrame`
        - 팩터 성과/랭크 테이블
        - 주요 컬럼: `["factorAbbreviation","factorName","styleName","cagr","rank_style","rank_total"]`

`_get_wgt(factor_rets: pd.DataFrame, data_raw: pd.DataFrame, data_neg: pd.DataFrame)`

`-> Tuple[df_mix, ports, main_factor, main_w, sub_factor, sub_w]`

- **Input**
    - `factor_rets`: 월간 팩터 수익률 행렬(열=팩터)
    - `data_raw`: 메인 팩터 1개를 담은 1행짜리 DataFrame
        - **필수 컬럼**: `["factorAbbreviation"]`
    - `data_neg`: downside 상관행렬(= `_generate_meta`의 `negative_corr`)
- **Output**
    - `df_mix: pd.DataFrame`
        - 메인–서브 조합별(서브 후보×가중치 grid) 성과 및 랭킹 테이블
        - 주요 컬럼: `main_wgt, sub_wgt, mix_cagr, mix_mdd, main_factor, sub_factor, rank_total ...`
    - `ports: List[pd.Series]`
        - 각 grid 포인트에 해당하는 mix return 시계열(= df_mix 행과 대응)
    - `main_factor: str`, `main_w: float`, `sub_factor: str`, `sub_w: float`
        - 최적 조합 및 비중

`assemble_top_style_portfolios(factor_rets, meta, neg_corr) -> Tuple[pd.DataFrame, pd.DataFrame]`

- **Input**
    - `factor_rets`: 월간 팩터 수익률 행렬
    - `meta`: 팩터 성과/랭크 테이블(상위 팩터 정렬 반영)
    - `neg_corr`: downside 상관행렬
- **Output**
    - `style_df: pd.DataFrame`
        - 스타일별 대표 포트폴리오(2-팩터 믹스) 수익률 시계열(열=스타일 태그)
    - `style_neg_corr: pd.DataFrame`
        - 스타일 포트폴리오 간 downside 상관행렬

`random_style_capped_sim(rtn_df, style_list, num_sims=1_000_000, style_cap=0.25, tol=1e-12)`

`-> Tuple[best_stats, weights_tbl]`

- **Input**

    - `rtn_df`: 월간 수익률 행렬(행=월, 열=팩터; 최종 후보 팩터 subset)
    - `style_list`: `rtn_df` 컬럼과 동일 순서의 스타일명 리스트
    - `num_sims`: 랜덤 포트폴리오 샘플 수
    - `style_cap`: 스타일별 최대 비중(기본 25%)
    - `tol`: 수치 허용오차
- **Output**
    - `best_stats: pd.DataFrame (1×...)`
        - 최적 포트폴리오의 성과 요약 (`cagr`, `mdd`, `rank_*`)
    - `weights_tbl: pd.DataFrame`
        - 최적 포트폴리오의 팩터 비중 테이블
        - 주요 컬럼: `["factor","raw_weight","styleName","fitted_weight"]`

`mp(start_date, end_date) -> None`

- **Input**
    - `start_date`, `end_date`: 문자열(날짜)
        - parquet 파일명 및 최종 리밸런싱 시점(`end_date`) 필터에 사용
- **Output**
    - 반환값: `None`
    - **Side Effects (파일 저장)**
        - 팩터/스타일/MP 비중 패널 및 피벗 결과 CSV 저장
        - 예시(코드 기준):
            - `aggregated_weights_{end_date}_test.csv`
            - `total_aggregated_weights_{end_date}_test.csv`
            - `total_aggregated_weights_style_{end_date}_test.csv`
            - `pivoted_total_agg_wgt_{end_date}.csv`
- **주요 내부 흐름(요약)**
    1. parquet + factor_info 로드 및 병합
    2. `_assign_factor`로 팩터별 분위/스프레드/원천 데이터 생성
    3. `_filter_grouped`로 섹터 필터 + 롱/숏 라벨링
    4. `_generate_meta`로 월간 수익률 행렬/랭킹/하락상관 생성
    5. `_get_wgt`로 스타일별 메인-서브 팩터 믹스 후보 생성
    6. `random_style_capped_sim`로 스타일 캡 제약 하 비중 최적화
    7. 종목별 최종 비중 산출 및 MP 집계, CSV 출력